1 Appendix // Mockup doodles // design sketches

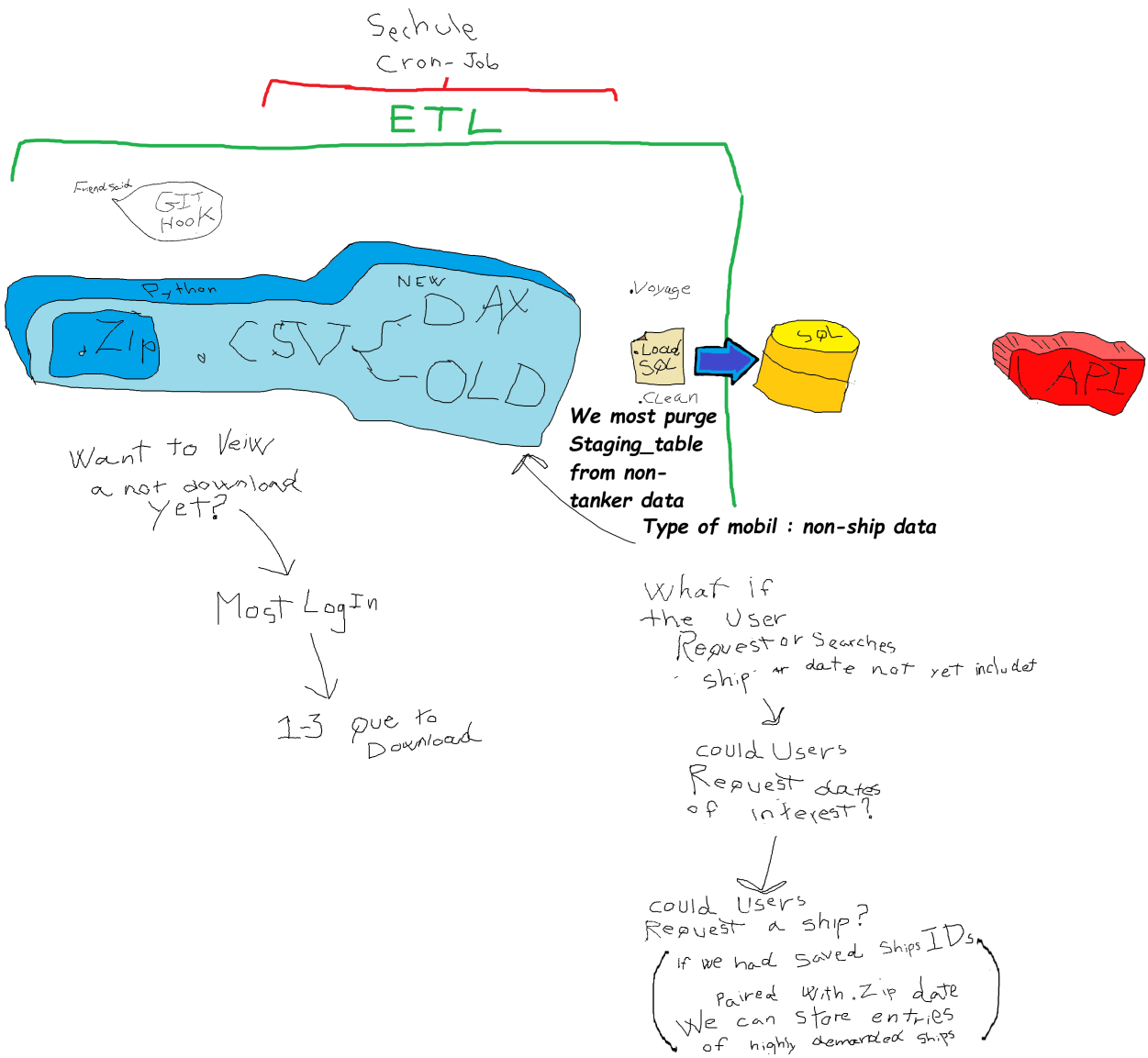

Figure 1: **Early architecture brainstorm.** First-pass sketch of the end-to-end flow: a scheduled (cron) job pulls zipped/CSV AIS dumps through a Python loader into a staging area, loads and cleans them into PostgreSQL, and serves the result through an API.

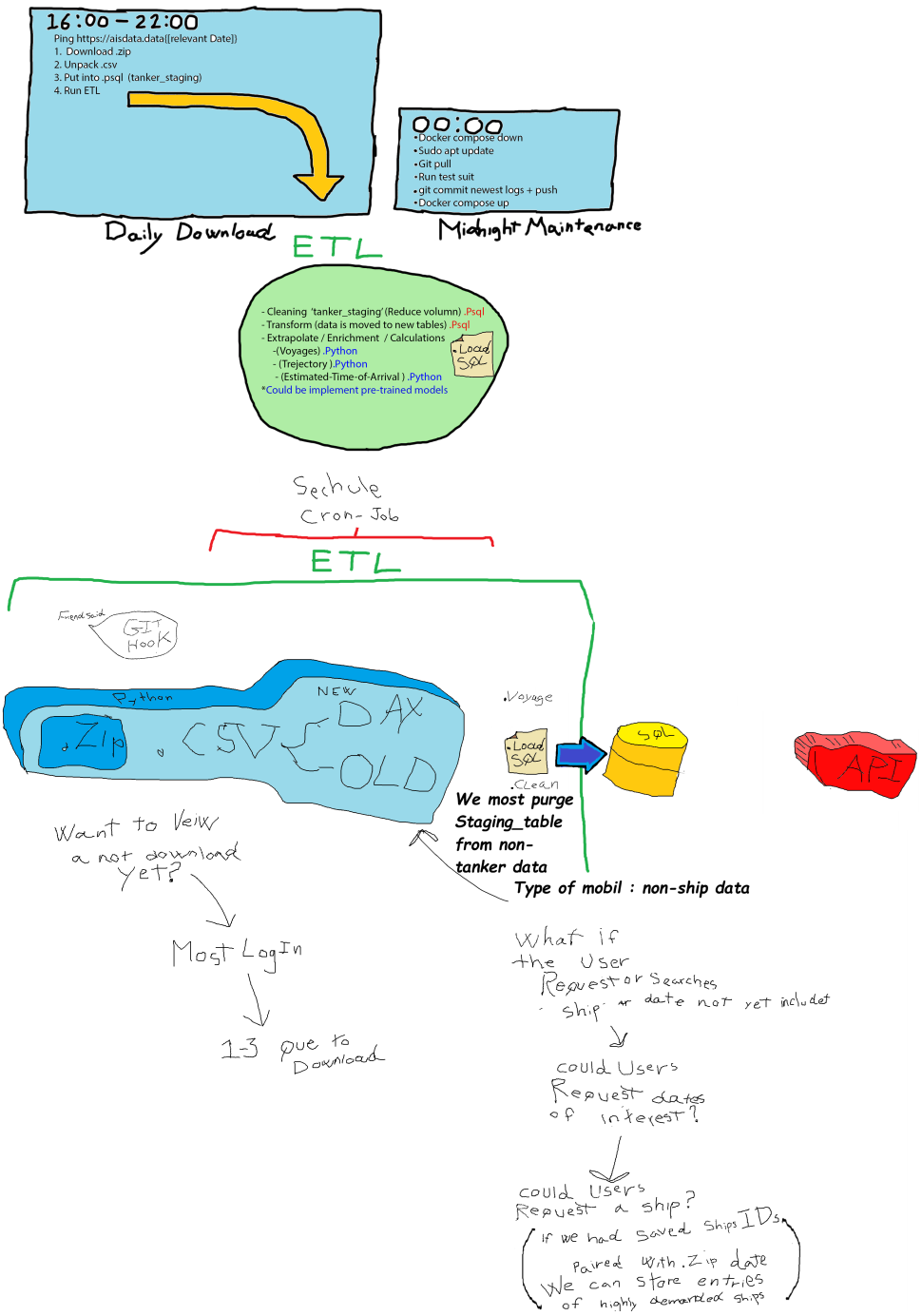

Figure 2: **Refined architecture sketch.** A later revision that folds the daily-download and midnight-maintenance schedules and the explicit ETL. This was when we used Cron-Job before Airflow was introduced.

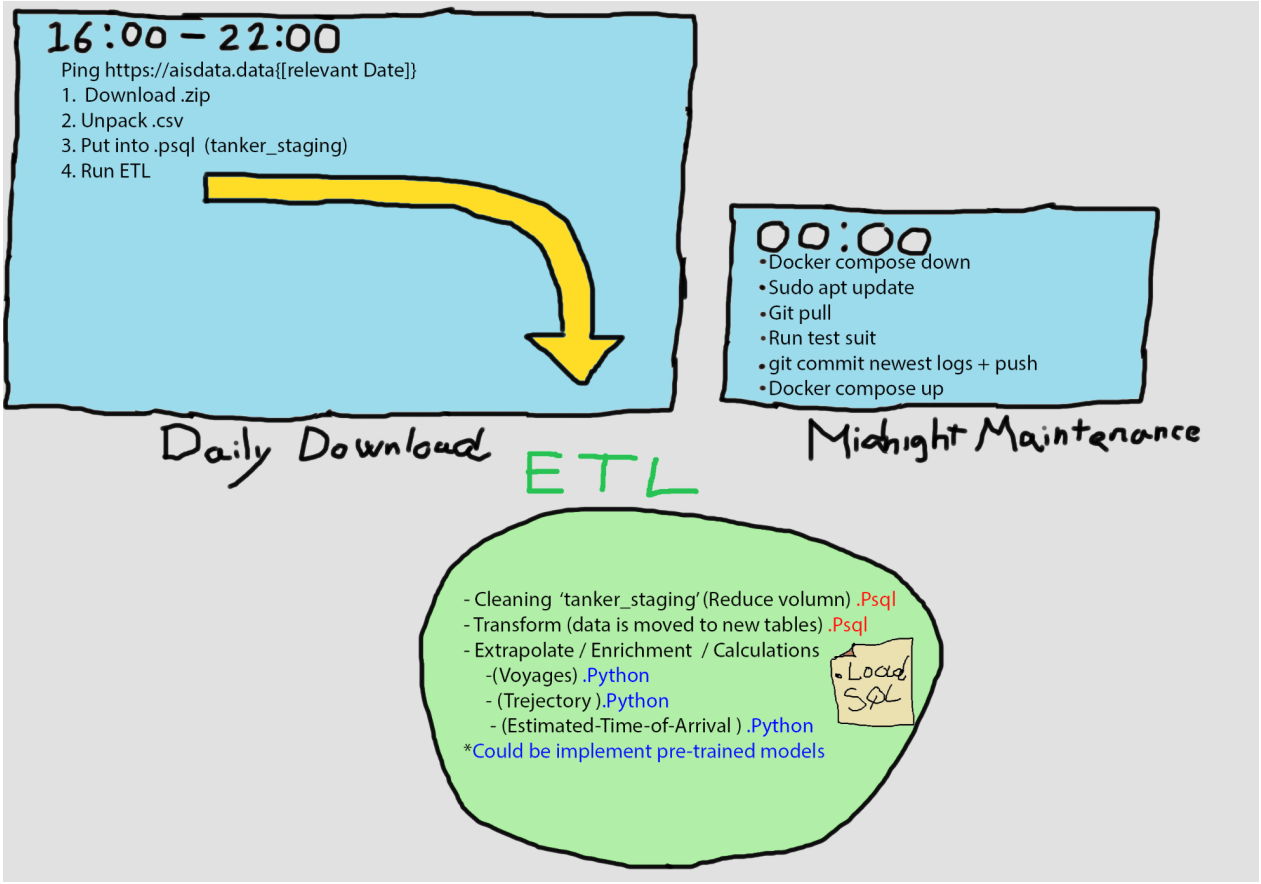

Figure 3: **Scheduled pipeline overview.** Early plans for our automation. This was before Airflow was introduced and we expected a our solution to be a combination of Cron-job and Bash and .sh files

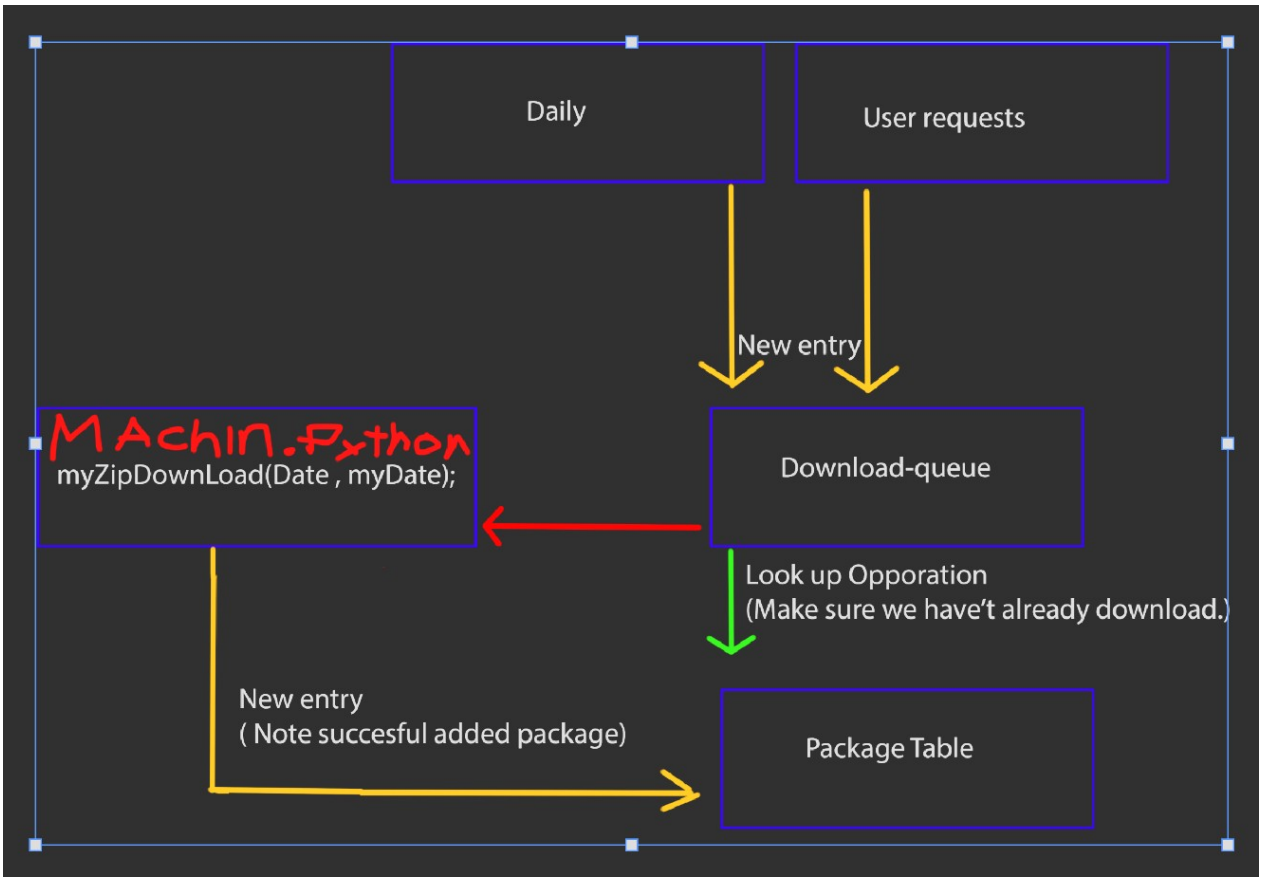

Figure 4: **Download-queue design.** Early plans for our design of data ingestion pipeline.

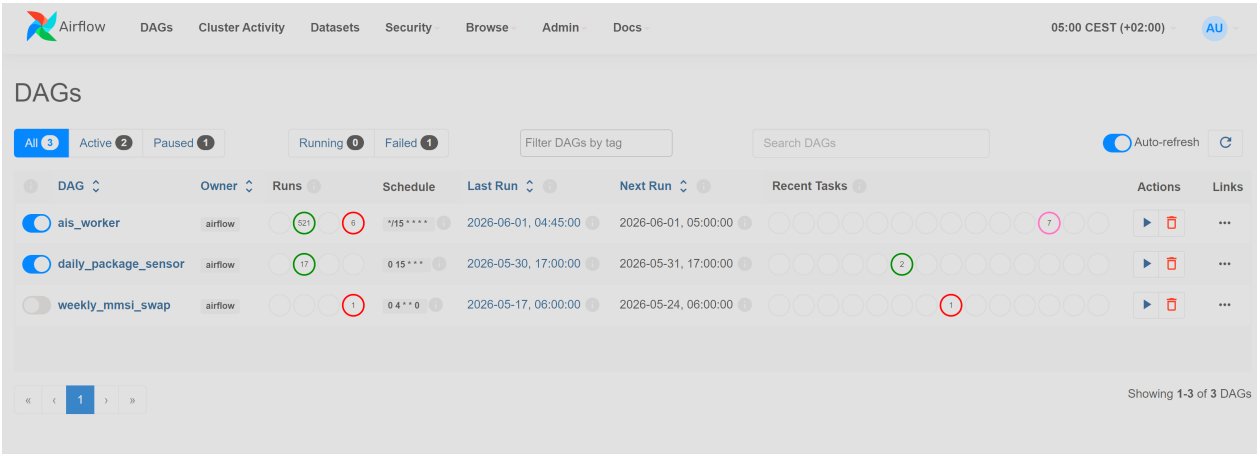

Figure 5: **Apache Airflow DAGs (implemented orchestration).** The scheduler that replaced the original cron sketch. `ais_worker` runs every 15 minutes to drain the consumer queue, `daily_package_sensor` runs daily at 15:00 to enqueue each day's batch, and the paused `weekly_mmsi_swap` runs Sundays at 04:00 for periodic MMSI maintenance. `521 successful runs`

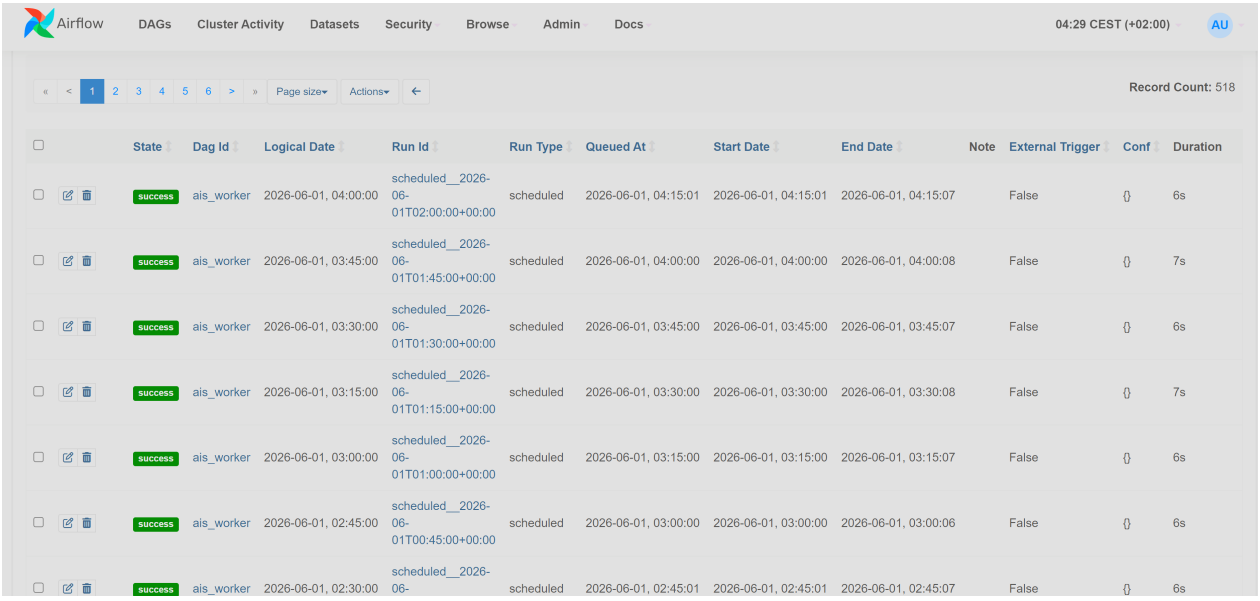

Figure 6: **Apache Airflow DAGs shown in a list**

```
ais_db=# SELECT * FROM data_consumer_queue ORDER BY queue_id DESC LIMIT 5;
 queue_id | source_batch_date | priority | requester | status  |          created_at
----------+-------------------+----------+-----------+---------+-------------------------------
        6 | 2026-05-06        |       10 | backfill  | pending | 2026-05-09 23:06:17.75808
        5 | 2026-05-05        |       10 | backfill  | pending | 2026-05-09 23:06:17.75808
        4 | 2026-05-04        |       10 | backfill  | pending | 2026-05-09 23:06:17.75808
        3 | 2026-05-03        |       10 | backfill  | pending | 2026-05-09 23:06:17.75808
        2 | 2026-05-02        |       10 | backfill  | pending | 2026-05-09 23:06:17.75808
(5 rows)
```

Figure 7: `data_consumer_queue` **contents.** Tail of the work queue that decouples *what* to fetch from *when* it is fetched. Each row pairs a `source_batch_date` with a `priority`, a `requester` (here `backfill`), and a `status`; the worker DAG processes pending rows in priority order.

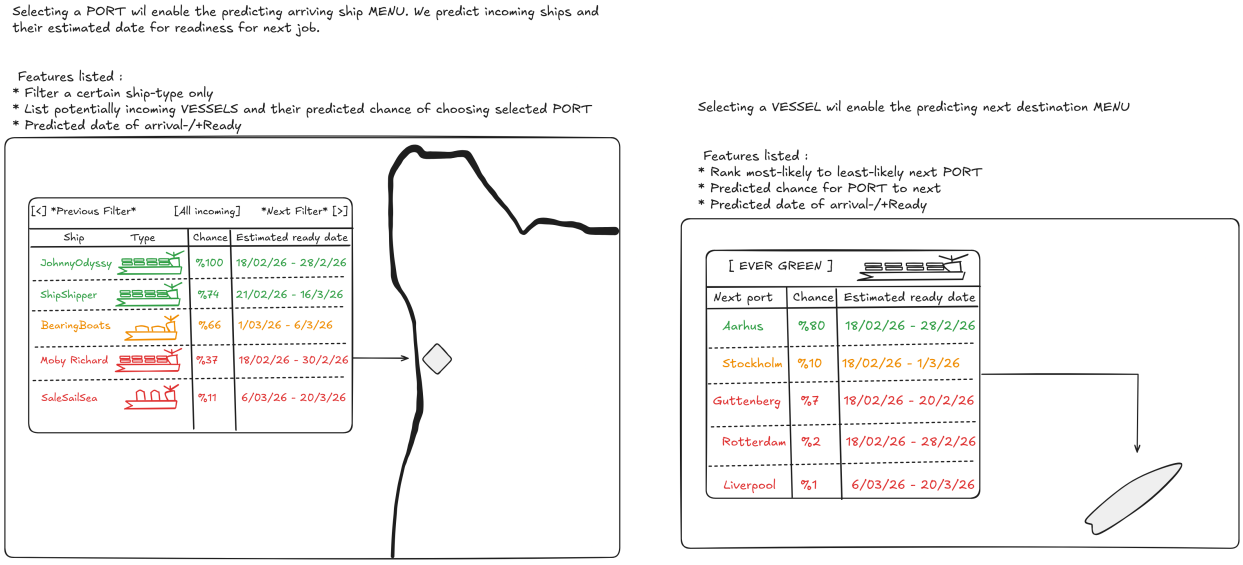

Figure 8: **Port and vessel-arrival prediction UI (cut content).** Mockup for a predictive feature that did not make the final scope. Selecting a port (left) would list incoming vessels with a predicted probability of choosing that port and an estimated readiness window; selecting a vessel (right) would rank its most-likely next ports with per-port probabilities and arrival estimates. This emerged due to a early plan of collaboration with Clarksons (Clarkson PLC) is the world's largest integrated shipping services and shipbroking group. Unfortunately, they stopped answering our emails, and we shifted our focus

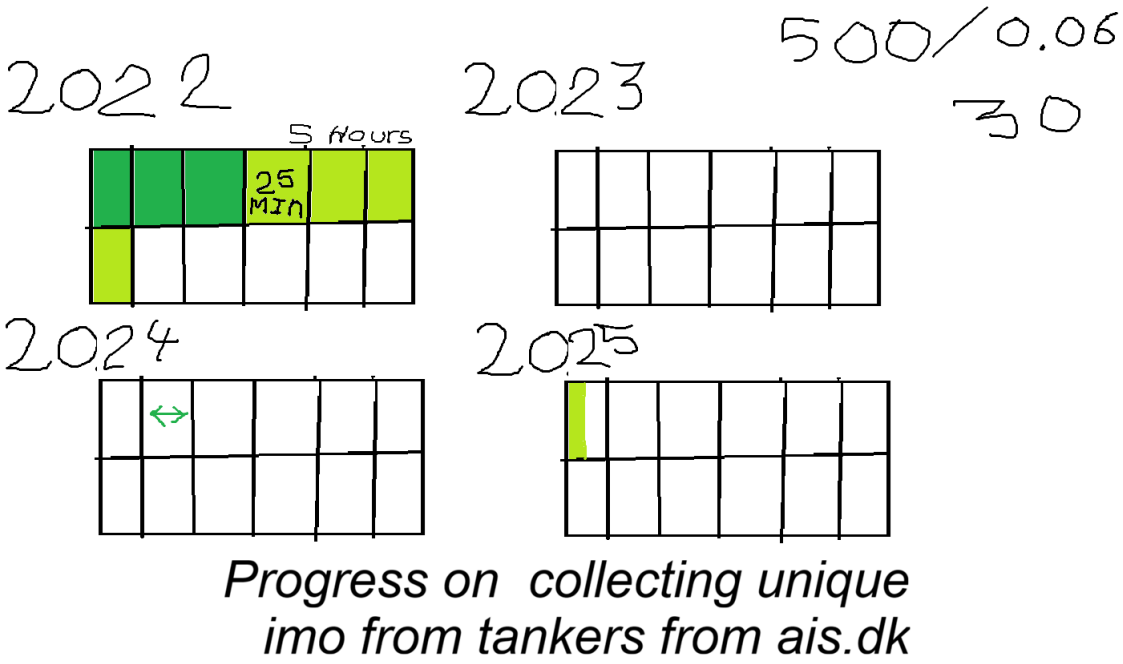

Figure 9: **IMO scanning.** We wasted a ton of time to scan packages for IMOs of tankers within the danish waters. This was because we wanted to pass this list on to our collaborator, Clarkson, in faith they would return more data on said tankers. This collaboration fell apart.